MACHAKOS UNIVERSITY

SCHOOL OF ENGINEERING AND TECHNOLOGY

BSC ELECTRICAL AND ELECTRONICS ENGINEERING

FINAL YEAR PROJECT REPORT

DESIGN AND IMPLEMENTATION OF A HIGH-SPEED VISIBLE LIGHT COMMUNICATION SYSTEM

DONE AND SUBMITTED BY:

TONY KIPCHIRCHIR

J26-6705-2020

SUPERVISOR:

MR. DUNCAN KILUNGU

DECLARATION

This project report is my original work and has not been presented for a degree or any other academic award at any other university or institution of higher learning.

Student Name: Tony Kipchirchir

Registration Number: J26-6705-2020

Degree: BSc Electrical & Electronic Engineering

Signature: _____ Date: _____

Approval

This project report has been submitted for examination with my approval as the university supervisor.

Supervisor Name: Mr. Duncan Kilungu

Signature: _____ Date: _____

DEDICATION

I dedicate this project to my family for their unwavering support, patience, and encouragement throughout my engineering studies. Their belief in my potential has been the driving force behind my academic journey. I also dedicate this work to all future researchers striving to democratize access to advanced telecommunications technology.

ACKNOWLEDGEMENT

The completion of this engineering project would not have been possible without the guidance, support, and resources provided by numerous individuals and institutions.

First and foremost, I wish to express my profound gratitude to my project supervisor, Mr. Duncan Kilungu, for their invaluable technical guidance, constructive criticism, and continuous mentorship throughout the research, design, and testing phases of this work.

I also extend my sincere appreciation to the faculty and technical staff of the Department of Electrical and Electronic Engineering for providing the foundational knowledge and laboratory facilities necessary to bring this hardware prototype to life.

Finally, I am deeply grateful to my peers and classmates for the collaborative environment and shared late-night troubleshooting sessions that ultimately made this project a success.

ABSTRACT

As wireless communication networks expand, the traditional radio frequency spectrum is experiencing severe congestion, leading to reduced data rates and increased latency. Furthermore, radio frequency signals easily penetrate walls, creating data security risks and posing electromagnetic interference hazards in sensitive environments such as hospitals and aircraft. Visible Light Communication offers a practical solution to these issues by utilizing the unlicensed visible light spectrum via standard indoor LED lighting, ensuring spatial data confinement and immunity to electromagnetic interference. However, current research is polarized between highly expensive Field Programmable Gate Array setups and severely constrained microcontroller systems limited to low-speed text transmission. This project addresses this gap by developing a low-cost, high-speed transceiver capable of streaming real-time High-Definition multimedia using discrete components within a strict budget of KES 23,500. The system utilizes a custom analog front-end featuring an OPA380 transimpedance amplifier and an LMV7219 digital comparator, bypassing expensive Analog-to-Digital Converters. A Raspberry Pi acts as the baseband processor, implementing a custom 4B5B line coding algorithm to achieve an 80 percent spectral efficiency and maintain direct-current balance for flicker-free illumination. To combat physical-layer noise, the application layer uses Motion-JPEG compression, enabling zero temporal error propagation. Empirical testing over distances of 10 to 200 centimetres validated the system performance, achieving a stable 1.17 megabits per second throughput up to 110 centimetres before experiencing the expected digital cliff attenuation. The results prove that accessible, discrete hardware paired with robust error-handling software can successfully democratize high-speed optical wireless network research.

Table of Contents

Table of Figures

List of Tables

LIST OF ABBREVIATIONS

Abbreviation	Full Meaning
4B5B	4-Bit to 5-Bit (Line Coding)
ADC	Analog-to-Digital Converter
AFE	Analog Front-End
AGC	Automatic Gain Control
APD	Avalanche Photodiode
BER	Bit Error Rate
BOM	Bill of Materials
CABAC	Context-Adaptive Binary Arithmetic Coding
CAVLC	Context-Adaptive Variable-Length Coding
CRC	Cyclic Redundancy Check
DC	Direct Current
DSP	Digital Signal Processor
EMI	Electromagnetic Interference
EOI	End of Image
FEC	Forward Error Correction
FFmpeg	Fast Forward Moving Picture Experts Group (Multimedia Framework)
FPGA	Field Programmable Gate Array
fps	Frames Per Second
GPIO	General-Purpose Input/Output
HD	High-Definition
IM/DD	Intensity Modulation and Direct Detection
IoT	Internet of Things
IR	Infrared
JPEG	Joint Photographic Experts Group
kbps	Kilobits Per Second

KES	Kenya Shillings
LED	Light Emitting Diode
Li-Fi	Light Fidelity
LoS	Line-of-Sight
MAC	Medium Access Control
Mbps	Megabits Per Second
MOSFET	Metal-Oxide-Semiconductor Field-Effect Transistor
M-JPEG	Motion-JPEG
NFC	Near Field Communication
OFDM	Orthogonal Frequency-Division Multiplexing
OOK	On-Off Keying
OWC	Optical Wireless Communications
PAM	Pulse Amplitude Modulation
PER	Packet Error Rate
PHY	Physical (Layer)
PIN	Positive-Intrinsic-Negative (Photodiode)
PRBS	Pseudo-Random Binary Sequence
PWM	Pulse Width Modulation
RC	Resistor-Capacitor (Time Constant)
RF	Radio Frequency
RMS	Root Mean Square
RX	Receiver Subsystem
SMPS	Switched-Mode Power Supply
SNR	Signal-to-Noise Ratio
TCP/IP	Transmission Control Protocol / Internet Protocol
TIA	Transimpedance Amplifier
TX	Transmitter Subsystem
UART	Universal Asynchronous Receiver-Transmitter

UAV	Unmanned Aerial Vehicle
V2V	Vehicle-to-Vehicle
VGA	Variable Gain Amplifier
VLC	Visible Light Communication
WDM	Wavelength Division Multiplexing

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

The exponential growth of wireless communication technologies has fundamentally revolutionized how data, voice, and multimedia content are transmitted across the globe. As systems evolve, researchers are exploring innovative frontiers to expand connectivity, including UAV array-aided visible light communication [1] and hybrid NFC-VLC systems for secure, localized integration [2]. At the core of this technological shift is the urgent need to address the severe congestion crisis in the traditional radio-frequency (RF) spectrum. As billions of Internet of Things (IoT) devices, mobile endpoints, and autonomous systems compete for limited, heavily regulated bandwidth, the result is signal overlap, increased latency, and hard physical data ceilings. The Shannon-Hartley theorem dictates that the capacity of these channels cannot scale indefinitely without proportional increases in bandwidth, which the RF spectrum can no longer comfortably provide.

Visible Light Communication (VLC), commercially referred to as Li-Fi (Light Fidelity), has emerged as a highly promising complementary technology to address these critical limitations. Recent performance analyses of indoor VLC systems utilizing diverse LED configurations and photodetectors demonstrate massive potential for high-speed, localized networking [3]. The technology is highly adaptable, with active surveys highlighting its expanding role in vehicular methodologies and intelligent transportation [4]. Rather than completely replacing traditional systems, VLC is increasingly viewed through the lens of hybrid VLC and RF networks, providing a necessary, complementary layer to handle high-bandwidth traffic and alleviate sub-6 GHz congestion [5].

Operating in the unlicensed visible light spectrum (400 to 800 THz), VLC offers thousands of times more bandwidth than the entire regulated RF spectrum. Because light waves cannot penetrate opaque barriers, VLC signals are absolutely confined to the physical geometry of the room, providing inherent, hardware-level security that cannot be intercepted from outside. Additionally, the technology leverages existing indoor LED lighting infrastructure to offer a dual-utility paradigm that provides both standard room illumination and high-speed data communication simultaneously.

1.2 Problem Statement

Modern wireless networks rely heavily on the radio frequency spectrum, which is becoming increasingly congested as more devices connect to the internet. This congestion reduces available bandwidth and slows down data transfer speeds. Additionally, radio frequency signals naturally pass through walls, making it difficult to secure confidential data in localized environments. These signals can also cause dangerous electromagnetic interference in sensitive areas like hospital surgical wards or aviation environments. Visible Light Communication solves these problems

because light cannot pass through walls, inherently securing the data, and it does not interfere with medical equipment. However, a major challenge exists in current Visible Light Communication research. Existing high-speed prototypes are usually built using highly expensive Field Programmable Gate Arrays that are inaccessible to undergraduate researchers. Conversely, affordable prototypes are typically built using simple microcontrollers like the Arduino, which are only fast enough to send text or audio. Therefore, the core problem is the lack of a low-cost, intermediate hardware architecture that can achieve the megabit-per-second speeds required for real-time video streaming without relying on expensive, enterprise-grade processing equipment.

1.3 Significance and Justification of the Study

This research holds significant value across multiple engineering disciplines by transitioning VLC from theoretical potential to practical deployment.

First, it champions hardware democratization. By conceptualizing and utilizing a discrete Analog Front-End (AFE) consisting of an OPA380 transimpedance amplifier, an LMV7219 nanosecond comparator, and a TC4420 MOSFET gate driver, this project demonstrates that complex and expensive Analog-to-Digital Converters (ADCs) can be entirely bypassed. The system achieves a robust 2 Mbps physical link capable of sustaining 1.17 Mbps of verified payload throughput for a total budget of approximately KES 23,500. This makes advanced telecommunications research highly accessible to educational institutions.

Second, the study justifies a cross-layer architectural approach to overcoming optical noise. By aggressively replacing the H.264 codec with Motion-JPEG (M-JPEG) over the optical link, the system achieves "Zero Temporal Propagation." If an optical packet is dropped due to ambient noise or a blocked line-of-sight, only a single independent frame is lost. The stream mathematically heals instantly in a fraction of a second, demonstrating how Application Layer logic can solve Physical Layer constraints.

Third, to maximize the hardware limits of the embedded UART interface, the project implements high-efficiency 4-Bit to 5-Bit (4B5B) line coding. This increases the theoretical payload capacity from the 50 percent efficiency of standard Manchester coding up to 80 percent efficiency. Furthermore, 4B5B guarantees enough voltage transitions to maintain strict DC-balance, ensuring the 10W transmission LED provides constant, flicker-free illumination to the human eye.

Finally, the project validates a communication medium that is inherently immune to Electromagnetic Interference (EMI). It provides a highly functional, tangible blueprint for wireless data deployment in RF-restricted zones such as hospital MRI wards, intelligent transportation systems, and aerospace applications.

1.4 Objectives

1.4.1 Main Objective

To design, implement, and validate a high-speed, low-cost Visible Light Communication (VLC) transceiver system capable of streaming real-time High-Definition video wirelessly over a 1.0–2.0 m indoor optical link.

1.4.2 Specific Objectives

1. To design a discrete optical analog front-end, integrating a TC4420-driven 10W LED transmitter and an OPA380 transimpedance amplifier with an LMV7219 voltage comparator to achieve nanosecond signal quantization without the use of an ADC.
2. To implement deterministic baseband processing on a Raspberry Pi 4 embedded platform, utilizing Application-to-Physical layer 4B5B line coding to maximize channel efficiency and maintain absolute DC-balance for flicker mitigation.
3. To achieve robust, real-time video transmission by deploying an M-JPEG processing pipeline, effectively eliminating the temporal error propagation associated with standard inter-frame codecs over noisy free-space optical channels.
4. To characterize the physical limitations of the optical link, mathematically analyzing the system's Throughput, Bit Error Rate (BER), and Packet Error Rate (PER) to identify the "Digital Cliff" threshold governed by the Inverse Square Law and ambient lux interference.

1.5 Scope and Limitations

Scope

This project focuses strictly on the Physical (PHY) and Data Link layers of a simplex (one-way) VLC system. The scope encompasses the engineering design of the discrete transmitter and receiver circuits, the C++ software implementation of the 4B5B encoding/decoding and CRC-16 error detection algorithms, and the integration of the FFmpeg multimedia framework for M-JPEG video scaling and processing. Empirical testing is limited to indoor environments under standard ambient lighting conditions (approximately 90 Lux) to evaluate throughput limits, path loss, and signal degradation matrices.

Limitations

1. **Line-of-Sight Dependency:** The fundamental physics of the VLC system require strict, uninterrupted line-of-sight between the transmitting LED and the receiving photodiode. Physical obstructions will completely sever the communication link.
2. **The Digital Cliff and Distance Constraints:** Due to the architectural decision to use a digital voltage comparator (LMV7219) rather than a continuous ADC to reduce processing latency, the system is subject to the "Digital Cliff" effect. The optical link maintains near-perfect throughput up to 1.6 meters. Beyond this point, the attenuated analog voltage drops below the fixed comparator threshold, resulting in total packet failure rather than graceful degradation.

3. **Lambertian Radiation and Angular Misalignment:** High-power LEDs naturally exhibit a Lambertian radiation pattern, meaning the emitted optical power drops off rapidly with the cosine of the viewing angle. Consequently, the system is highly sensitive to angular misalignment and requires fixed, stationary positioning for optimal throughput, lacking the omnidirectional flexibility of standard Wi-Fi.
4. **Simplex Operation:** The current architecture supports point-to-point, single-direction transmission. Bi-directional (duplex) communication, uplink feedback channels, and multi-user network access (MAC layer protocols) are beyond the scope of this physical-layer hardware implementation.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction and Theory

The evolution of wireless networks has continually pushed the boundaries of the electromagnetic spectrum. As the telecommunications industry aggressively investigates technologies for 5G and beyond, Optical Wireless Communications (OWC) has emerged as a cornerstone of future network architectures [9].

The foundational theory of Visible Light Communication (VLC) rests on Intensity Modulation and Direct Detection (IM/DD). Unlike traditional radio frequency systems that rely on the coherent modulation of signal phase and amplitude, VLC relies on the incoherent nature of LEDs. In the IM/DD paradigm, a digital baseband signal is superimposed directly onto the forward driving current of a Light Emitting Diode. The resulting high-frequency variations in luminous intensity are captured by a photodetector and converted back into electrical photocurrent.

The theoretical maximum data rate of any communication channel affected by noise is defined by the Shannon-Hartley theorem [12]. In the radio frequency domain, channel capacity is severely restricted by tight governmental bandwidth allocations. Conversely, the visible light spectrum spans approximately 400 terahertz of unlicensed bandwidth. This massive spectral availability allows VLC systems to theoretically achieve multi-gigabit data rates [10].

However, translating this theoretical channel capacity into functional hardware introduces complex signal processing requirements governed by optical physics. The direct current (DC) channel gain of an optical link is highly dependent on the Lambertian emission pattern of the transmitting LED [13]. As light leaves the semiconductor die, it diverges in a conical pattern. The optical power that successfully reaches the receiver decreases exponentially according to the Inverse Square Law. Furthermore, modern VLC engineering is entirely constrained by three physical factors: the parasitic capacitance of the receiving photodiode, the thermal noise floor of the transimpedance amplifier, and the computational efficiency of the baseband encoding algorithms utilized to combat channel attenuation.

2.2 Current Solutions

The current landscape of VLC research demonstrates a rapid expansion of the technology across highly diverse application environments. Researchers are no longer confining VLC to strictly controlled indoor office environments but are pushing it into highly dynamic and unstable networking scenarios.

2.2.1 Mobility and Spatial Diversity

In mobility and outdoor applications, Al Hasnawi and Marghescu surveyed vehicular VLC methodologies, demonstrating how standard LED headlights and taillights can facilitate high-speed vehicle-to-vehicle (V2V) safety networks [4]. This concept is being further expanded into three-dimensional networking spaces. Recent studies by Zhang et al. utilized UAV array-aided VLC to enhance angle diversity, proving that multi-directional optical transmitters can

maintain robust optical links between flying drones even under significant mechanical vibration [1]. To maximize transmission distance in these harsh vehicular and outdoor contexts, researchers have successfully implemented aggressive techniques such as LED current overdriving, pairing it with modified variable pulse position modulation to force optical signals through adverse atmospheric conditions like fog and rain [6].

2.2.2 Hybrid Architectures

To address the inherent line-of-sight (LoS) limitations of optical networks, significant research has pivoted toward hybrid architectures that utilize both light and radio waves. Bravo-Alvarez et al. reviewed hybrid VLC/RF networks, showcasing how smart systems can seamlessly hand off data streams to standard Wi-Fi or cellular networks the moment the optical link is physically blocked by an obstruction [5]. On a more localized, low-power scale, Jayaweera et al. successfully explored hybrid NFC-VLC systems. In this architecture, Near Field Communication is used to establish secure, close-proximity handshakes for IoT devices, which then transition to the VLC channel for high-bandwidth data payloads [2].

On the physical hardware front, the transmission boundaries are constantly being tested. Elamrawy et al. conducted exhaustive performance analyses on indoor systems by testing diverse photodetector geometries and LED configurations to optimize the Signal-to-Noise Ratio (SNR) across varying room dimensions [3].

2.3 Critical Analysis

While the current literature showcases the incredible versatility of visible light networks, a critical engineering analysis reveals massive disparities in how these systems are physically implemented. Existing solutions largely ignore the economic realities and processing limitations of embedded systems, creating a significant barrier to practical, real-world adoption.

2.3.1 Hardware and Analog Front-End Limitations

The most complex bottleneck in any optical receiver is the Transimpedance Amplifier (TIA). The TIA is responsible for converting the microscopic, nano-ampere photocurrent generated by the receiving photodiode into a measurable analog voltage. Kavlak and Ayten analyzed TIA design constraints, concluding that achieving high bandwidth and high gain simultaneously requires highly specialized, precision-tuned circuits [7]. The physical limitation stems from the photodiode's junction capacitance. When paired with a high-value feedback resistor in the amplifier circuit, this capacitance creates an RC time constant that severely limits the frequency response of the receiver.

In academic literature, this analog challenge is frequently bypassed by throwing massive budgets at the problem. Researchers utilize commercially available optical receivers paired with high-speed Analog-to-Digital Converters (ADCs) sampling at millions of times per second. While ADCs allow engineers to process complex analog waveforms in the digital domain using Digital Signal Processors (DSPs), they introduce significant processing latency and require massive computational power. There is a distinct lack of literature detailing discrete, low-cost analog front-ends that rely on high-speed digital voltage comparators to achieve nanosecond signal quantization without relying on an ADC bottleneck.

2.3.2 Baseband Processing and Line Coding

To push high data rates through standard LEDs, researchers often turn to complex digital modulation schemes. Othman et al. evaluated hybrid Pulse Amplitude Modulation (PAM) and Pulse Width Modulation (PWM) to increase spectral efficiency [8]. While PAM allows multiple bits to be transmitted per optical symbol by varying the brightness levels, it drastically shrinks the margin for error. Decoding subtle changes in brightness requires a continuous, high-resolution ADC, completely disqualifying PAM for low-cost embedded hardware like a Raspberry Pi.

Furthermore, basic unipolar encoding schemes (such as standard UART transmission) fail to maintain DC-balance. This causes the transmission LED to flicker visibly to the human eye during long strings of binary zeros, violating the dual-utility mandate of VLC. The official IEEE 802.15.7 standard for Short-Range Optical Communication defines Manchester encoding to solve this flicker [14]. Manchester forces a voltage transition in the middle of every bit, ensuring perfect DC-balance. However, it is mathematically inefficient, operating at exactly 50 percent efficiency. It requires 2 megahertz of hardware bandwidth to transmit 1 megabit of data. The literature rarely addresses the implementation of highly efficient line coding borrowed from fiber optics, such as 4B5B coding, which can achieve 80 percent efficiency while preserving DC-balance on low-power embedded systems.

2.3.3 Video Compression and Temporal Propagation

The most glaring omission in current low-cost VLC literature is the handling of High-Definition multimedia. Accessible analog front-end transceivers documented in recent studies, such as the architecture proposed by Fuada et al., are strictly limited to transmitting real-time audio or basic text [11].

When video transmission is attempted, researchers typically rely on standard inter-frame codecs like H.264. H.264 utilizes delta frames (P-frames) that rely on temporal references to previous frames to save bandwidth. In a physical optical channel heavily influenced by ambient light noise and the Inverse Square Law, single-bit errors are statistically inevitable. H.264 utilizes Variable Length Coding (CABAC/CAVLC) for entropy compression. When a single bit flips in an optical stream, the decoder misinterprets the length of that symbol, instantly losing byte alignment. The compression reference breaks, and the visual distortion propagates temporally across the screen for several seconds.

A far more robust architectural approach involves relying on intra-frame compression, such as Motion-JPEG (M-JPEG) [15]. Because M-JPEG compresses each frame completely independently, it achieves "zero temporal propagation." If the optical link drops a packet due to a comparator threshold failure, only a single frame is corrupted, and the video stream mathematically heals itself in a fraction of a second.

2.4 The Research Gap

A comprehensive review of the literature reveals a distinct "missing middle" in Optical Wireless Communications.

At the high end of the spectrum, academic researchers achieve multi-gigabit speeds by utilizing Field Programmable Gate Arrays (FPGAs), Avalanche Photodiodes (APDs), and complex Orthogonal Frequency-Division Multiplexing (OFDM) modulators. At the low end

of the spectrum, hobbyist and educational implementations utilize basic 8-bit microcontrollers capable of pushing only a few kilobits per second for audio or text transfer [11].

Currently, no documented, highly detailed architecture successfully bridges this gap to provide real-time High-Definition video streaming on a restrictive budget. Specifically, the literature lacks a discrete, embedded Linux-based transceiver that combines a TC4420-driven LED transmitter with an OPA380 and LMV7219 receiver circuit to completely bypass ADC latency. Furthermore, the integration of 4B5B line coding to maximize embedded UART efficiency, coupled directly with an M-JPEG processing pipeline to eliminate temporal error propagation, represents a novel and unexplored systems-engineering approach in affordable VLC design.

This project directly addresses these gaps. It provides a complete empirical analysis of a highly optimized, low-budget VLC architecture capable of sustaining 1.17 megabits-per-second of verified payload throughput, fundamentally democratizing high-speed optical telecommunications research for resource-constrained environments.

CHAPTER 3: METHODOLOGY AND SYSTEM DESIGN

3.1 Introduction

The primary objective of this project was to design, implement, and characterize a low-cost, high-speed Visible Light Communication (VLC) system capable of real-time video transmission. To achieve this, the system architecture was partitioned into three highly optimized layers: a discrete analog front-end for physical optical transmission, an embedded Data Link layer for custom 4B5B line coding, and an Application layer utilizing M-JPEG video compression. The entire architecture was orchestrated using two Raspberry Pi 4B single-board computers acting as the transmitter and receiver basebands.

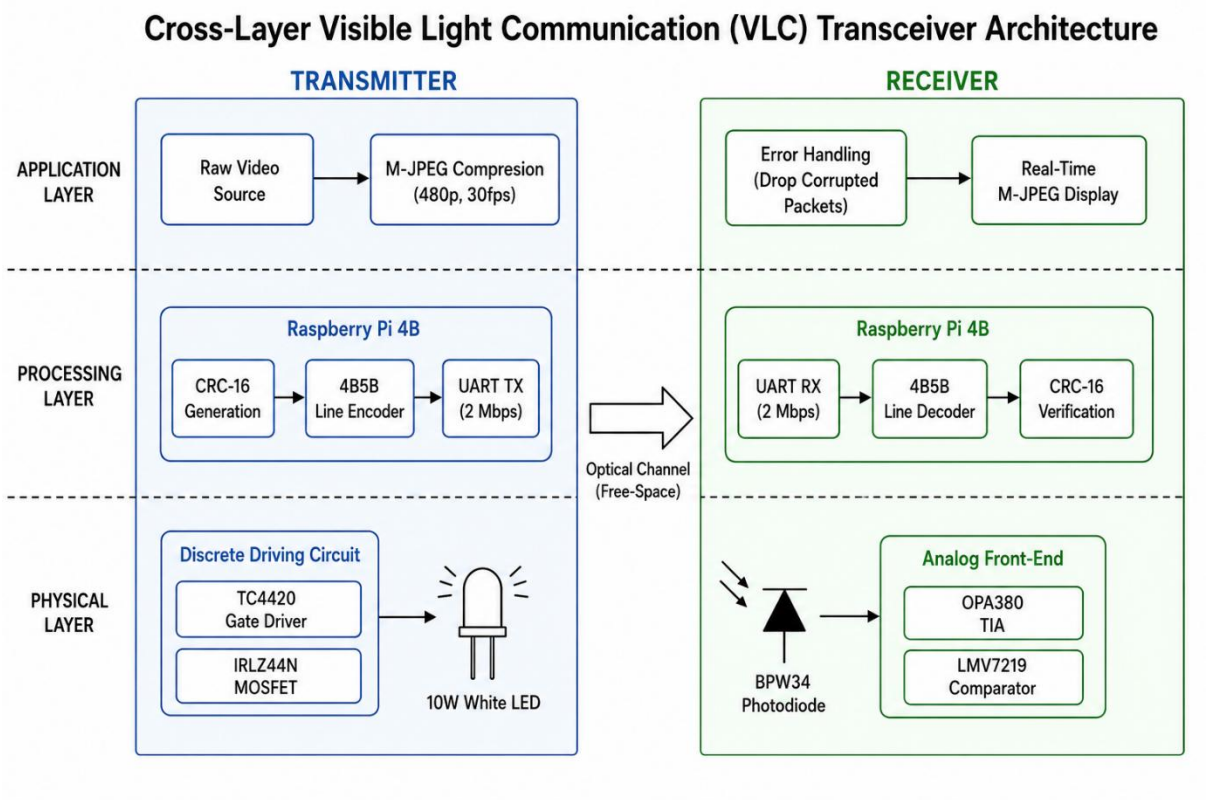

Figure 1: Cross-Layer VLC Transceiver Architecture

3.2 System Architecture Overview

The system operates as a simplex (unidirectional) optical link. The transmission pipeline begins with a raw video file processed by the transmitter's embedded Linux environment. The video is compressed, packetized, and sent to the hardware Universal Asynchronous Receiver-Transmitter (UART) interface at a baud rate of 2,000,000 bits per second (2 Mbps). A custom discrete driving circuit modulates a high-power 10W LED to physically transmit the digital baseband signal across the free-space optical channel.

At the receiver, a high-speed PIN photodiode captures the attenuated optical pulses. A custom analog front-end amplifies and quantizes these microscopic photocurrents back into 3.3V digital logic. The receiver's Raspberry Pi decodes the line coding, verifies the data integrity using a Cyclic Redundancy Check (CRC), and renders the video stream to the display in real time.

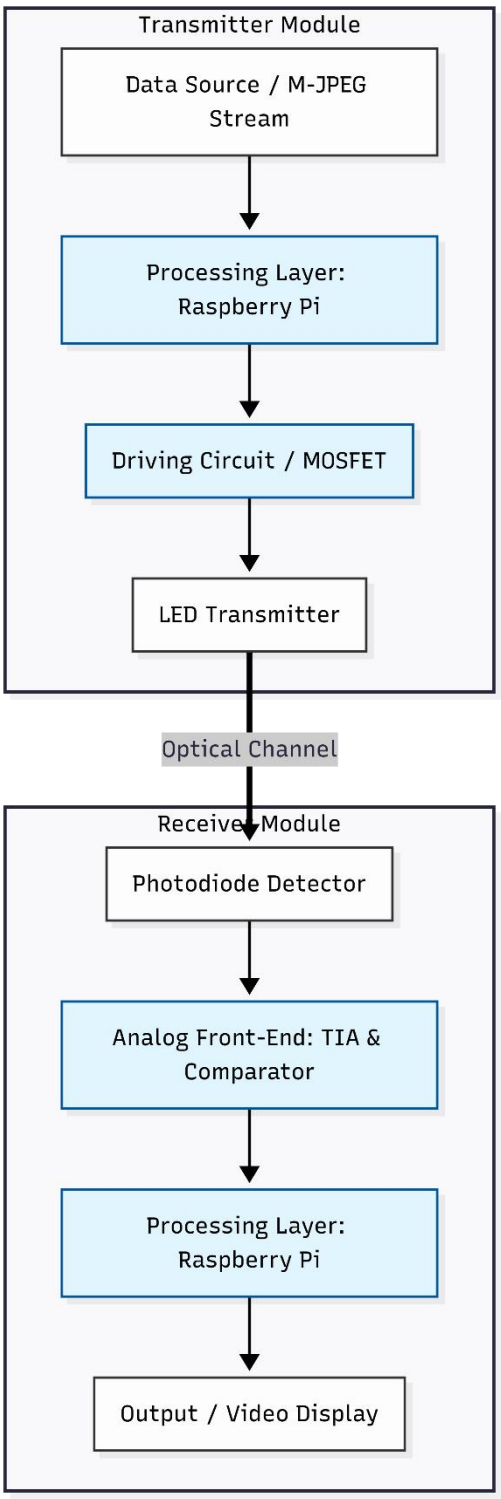

Figure 2: High-Level System Architecture

3.3 Transmitter Hardware Design

To physically transmit a 2 Mbps digital signal using a high-power LED, the baseband GPIO signals from the Raspberry Pi required significant current and voltage amplification. The transmitter circuit was designed as a cascade of power-switching stages consisting of the Raspberry Pi 4B, a TC4420 gate driver, an IRLZ44N power MOSFET, a 10W LED, a 10-ohm current-limiting resistor, and an LM2596 step-down power supply.

3.3.1 Power Delivery and Logic Translation

The illumination source is a standard commercial 10W white LED, which requires a stable 12V DC power supply to achieve maximum luminous intensity. To provide this, an LM2596 switch-mode step-down (buck) converter was utilized to regulate power from a higher-voltage mains adapter down to a highly stable 12V rail. A 10-ohm high-wattage resistor was placed in series with the LED to limit the forward current, preventing thermal runaway and protecting the semiconductor junction.

3.3.2 High-Speed MOSFET Switching

Switching a 12V, high-current load at 2 MHz presents a severe physical challenge. The Raspberry Pi 4B GPIO pins output a maximum of 3.3V and can source only 16 milliamperes of current. The switching component chosen to drive the LED was the IRLZ44N, a logic-level N-channel MOSFET. While the IRLZ44N can handle the high current demands of the 10W LED, it possesses a significant parasitic gate capacitance (approximately 1700 pF). If the Raspberry Pi attempted to drive this MOSFET directly, the low 16 mA current would take several microseconds to charge the gate capacitance. This would result in slow, rounded optical pulses, severely restricting the maximum bandwidth of the channel and completely failing at 2 Mbps.

To overcome this RC time constant bottleneck, a Microchip TC4420 high-speed MOSFET gate driver was placed between the Raspberry Pi and the IRLZ44N. The TC4420 acts as an aggressive current buffer. It takes the weak 3.3V logic signal from the Pi's UART TX pin and outputs a massive 6-ampere peak current to the gate of the IRLZ44N. This violently forces the MOSFET gate to charge and discharge in less than 25 nanoseconds, ensuring crisp, perfectly square optical pulses at the 10W LED.

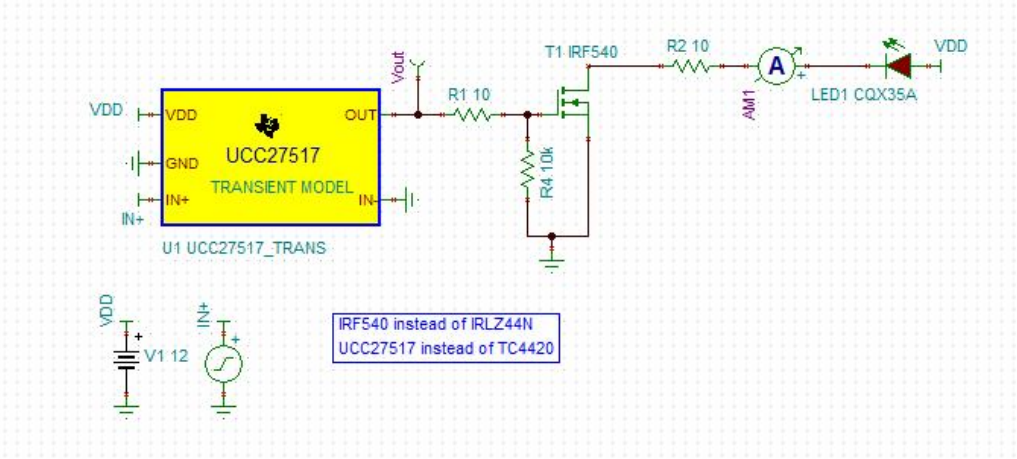

Figure 3: Circuit Schematic of the transmitter

3.4 Receiver Hardware Design

The receiver analog front-end is arguably the most critical and delicate subsystem of the VLC architecture. It must capture rapidly fluctuating light levels, filter out ambient room lighting, and convert nano-ampere currents into readable digital logic without introducing latency. The receiver chain consists of a BPW34 photodiode, an OPA380 transimpedance amplifier, an LMV7219 high-speed comparator, and the receiving Raspberry Pi 4B.

3.4.1 Optical Detection and Transimpedance Amplification

The optical signal is detected by a BPW34 PIN photodiode. The BPW34 was selected due to its wide spectral sensitivity and relatively low junction capacitance when reverse-biased, which is essential for high-frequency response. When the 10W LED pulses, the photodiode generates a microscopic photocurrent that is directly proportional to the received optical power.

To convert this current into a measurable voltage, the photodiode is connected to a Texas Instruments OPA380 precision Transimpedance Amplifier (TIA). The OPA380 is specifically designed for high-speed photodiode applications. It provides massive transimpedance gain (amplification) while maintaining a high bandwidth limit. This stage is crucial because the received optical power drops exponentially according to the Inverse Square Law [13]. At distances exceeding one meter, the OPA380 must extract an incredibly weak signal from the surrounding thermal and ambient noise floor.

3.4.2 Digital Quantization

Standard commercial VLC receivers often pass the TIA analog voltage into a high-speed Analog-to-Digital Converter (ADC) for software processing. To strictly minimize cost and computational latency, this project utilized a hardware digitization approach via a Texas Instruments LMV7219 comparator.

The LMV7219 is an ultra-high-speed comparator with a 7-nanosecond propagation delay. It continuously monitors the analog voltage output of the OPA380 against a predetermined reference voltage. When the amplified optical pulse exceeds the reference threshold, the LMV7219 instantly snaps its output to 3.3V (Logic 1). When the optical pulse falls below the threshold, it snaps to 0V (Logic 0). This crisp digital square wave is fed directly into the UART RX pin of the receiving Raspberry Pi 4B. This specific architectural choice creates a highly efficient system but inherently subjects the receiver to the "Digital Cliff" phenomenon, where the link will completely fail the moment the OPA380 voltage drops below the fixed comparator threshold.

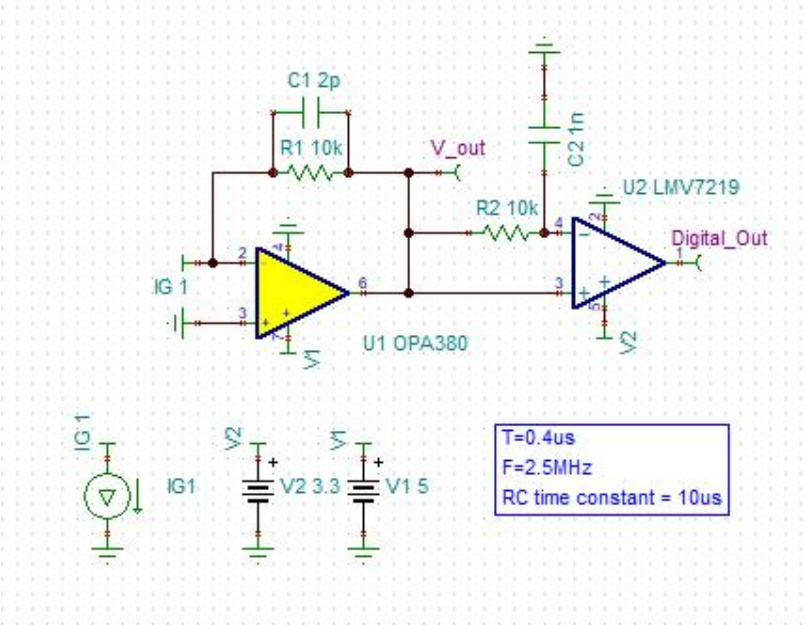

Figure 4: Circuit Schematic of the Receiver

3.5 Software and Protocol Design

With the physical layer established at a fixed baud rate of 2 Mbps, the embedded software stack was developed in C++ to handle data framing, error detection, and line coding.

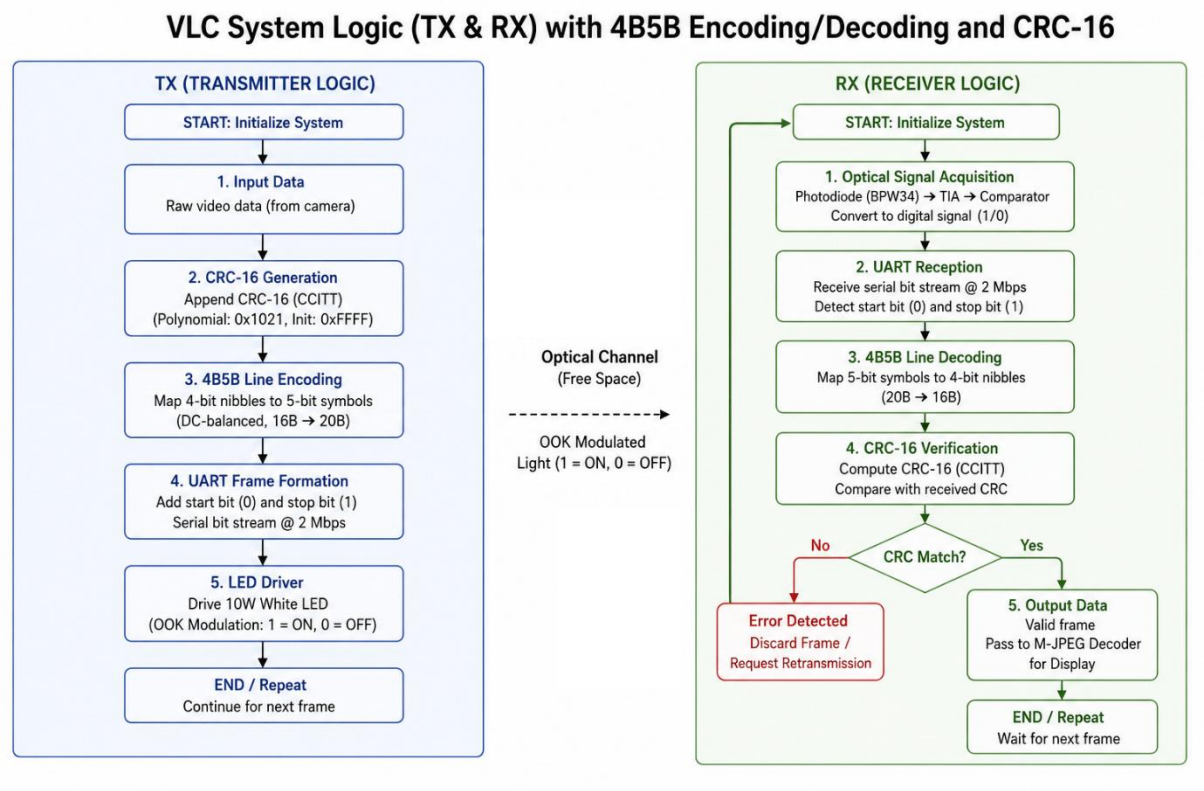

Figure 5: VLC System Logic

3.5.1 The 4B5B Line Coding Implementation

Transmitting raw binary data over an LED violates the VLC dual-utility requirement. Long strings of zeros would turn the LED off for noticeable fractions of a second, causing visible flicker. Furthermore, the receiving UART hardware requires frequent voltage transitions to maintain clock synchronization.

To solve this, the IEEE 802.15.7 VLC standard recommends DC-balanced line coding [14]. Instead of using the standard Manchester encoding (which has a poor 50 percent spectral efficiency), this project implemented custom 4-Bit to 5-Bit (4B5B) line coding. The C++ algorithm processes the video payload in 4-byte blocks. It slices these blocks into 4-bit nibbles and maps them to 5-bit symbols using the ANSI X3T9.5 standard table. These specific 5-bit symbols guarantee that no more than three consecutive zeros are ever transmitted.

This approach mathematically increases the channel efficiency to 80 percent. While the physical UART operates at 2,000,000 baud, the 4B5B compression allows a raw payload throughput of 1.6 Mbps (1,600,000 bits per second), which is highly sufficient for compressed video streaming.

3.5.2 Data Link Layer Packetization

To ensure data integrity, the raw video stream is packetized before optical transmission. Each packet consists of a total length of 68 bytes, heavily optimized for the 4B5B block alignment. The packet structure includes:

- **Length Byte (1 byte):** Specifies the size of the payload.
- **Sequence Byte (1 byte):** An incrementing counter used by the receiver to strictly track dropped or lost packets, enabling real-time Packet Error Rate (PER) calculation.
- **Video Payload (Up to 64 bytes):** The raw multimedia data.
- **CRC-16 (2 bytes):** A Cyclic Redundancy Check calculated over the entire packet to detect bit flips caused by optical channel noise.

Figure 6: Packet Structure Diagram

3.6 Application Layer: M-JPEG Video Compression

To fit High-Definition video into the 1.6 Mbps optical payload limit, rigorous Application Layer compression is required. As identified in the literature, standard inter-frame codecs (such as H.264) suffer catastrophic temporal error propagation in noisy optical environments.

A single bit error corrupts the mathematical reference frame, scrambling the video for several seconds.

To circumvent this, the system leverages the FFmpeg multimedia framework to implement Motion-JPEG (M-JPEG) compression. Governed by the JPEG still picture compression standard [15], M-JPEG is an intra-frame codec. It compresses every single frame of the video as an entirely independent, self-contained image. If an optical packet is corrupted by a physical obstruction or ambient noise, the receiver's CRC-16 algorithm drops the broken packet. The video player simply discards the single broken frame and instantly draws the next complete JPEG frame a fraction of a second later.

By scaling the video resolution to 480p (854x480 pixels) and tuning the quantization parameter, the M-JPEG stream was mathematically confined to exactly 1.17 Mbps. This created a highly robust "Zero Temporal Propagation" video stream that heals instantly from physical channel interruptions.

3.7 Experimental Setup and Data Collection Protocol

To evaluate the success of the architecture, an empirical data collection protocol was established. The transmitter and receiver modules were securely mounted on a linear track in an indoor laboratory setting with an ambient lighting baseline of 92 Lux.

A custom C++ benchmarking script was developed to replace the video stream with a "Golden Payload" consisting of known alternating bits (0x55). The transmitter continuously blasted this payload at 2 Mbps. The receiver recorded data over strict 3-minute intervals. The physical distance between the LED and the photodiode was increased in 10-centimeter increments, starting from 10 cm and concluding at 200 cm. For each distance interval, the receiver logged the total bits received, Bit Errors, calculated Bit Error Rate (BER), lost packets, Packet Error Rate (PER), and effective throughput in kilobits per second (kbps). This methodology was specifically designed to capture the "Digital Cliff" degradation curve governed by the Inverse Square Law.

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Introduction

This chapter presents a comprehensive empirical analysis of the Visible Light Communication (VLC) system's performance. Rather than probing physical analog waveforms, the system was characterized by evaluating the Data Link and Application layer metrics: Bit Error Rate (BER), Packet Error Rate (PER), and effective payload throughput (measured in kbps). The experimental setup maintained a constant ambient light baseline of 92 Lux. By systematically analyzing the system's performance across incremental distances from 10 cm to 200 cm, the testing protocol successfully mapped the transition of the optical channel from a highly deterministic state to a stochastic state, ultimately reaching total link failure.

4.2 Mathematical Throughput Validation

Before analyzing the error rates, it is crucial to validate the baseline throughput of the system mathematically. The physical hardware was clocked at a UART baud rate of 2,000,000 bits per second (2 Mbps). In standard UART transmission featuring one start bit and one stop bit, every 8-bit byte requires 10 bits on the wire. This yields a raw physical transmission limit of 200,000 bytes per second.

To maintain DC-balance, the system utilized 4B5B line coding, which maps 4 bytes of raw data to 5 bytes of transmitted data. This 80 percent efficiency reduces the line rate to 160,000 decoded bytes per second. Finally, the Data Link layer packetizes the data into 68-byte frames, containing 64 bytes of video payload and 4 bytes of control overhead (Sync, Length, Sequence, and CRC-16). This yields a payload efficiency of 94.1 percent.

Multiplying the decoded byte rate by the payload efficiency (160,000 bytes multiplied by 0.941) provides a theoretical maximum payload rate of 150,588 bytes per second, or roughly 1.20 Mbps. The empirical data recorded a stable maximum throughput of 1177 kbps (1.17 Mbps). This closely aligns with the mathematical limit, proving that the Raspberry Pi embedded software operates with near-perfect processing efficiency with minimal scheduling delays.

4.3 Tabulated Performance Data

Table 4.1 summarizes the observed software-layer metrics across key distance intervals using a continuous, known bitstream.

Table 1: Empirical Optical Link Performance Metrics

Distance (cm)	Throughput (kbps)	Bit Errors	BER (Log Scale)	PER (%)	Link Status

50	1177.00	0	1.0×10^{-10}	0.00	Highly Stable
100	1177.03	0	1.0×10^{-10}	0.00	Highly Stable
120	1176.72	67	3.16×10^{-7}	0.02	Marginal Degradation
150	1125.06	19,500	9.15×10^{-5}	4.35	Degradation "Knee"
160	878.74	150,000	7.08×10^{-4}	25.37	Severe Packet Loss
170	181.82	216,000	3.01×10^{-3}	84.55	Link Collapse
200	0.00	0	1.0×10^{0}	100.00	Total Failure

(Note: The BER value of 1.0×10^{-10} *at distances under 110 cm represents the baseline hardware noise floor of the system, where zero errors were recorded during the three-minute testing windows.)*

4.4 Zonal Performance Analysis

A detailed analysis of the data reveals three distinct operational zones governing the physical optical channel.

4.4.1 The Stability Zone (10 cm to 110 cm)

Within the first meter of transmission, the system exhibited flawless data integrity. The receiver recorded zero bit errors and maintained a rock-solid throughput averaging 1177 kbps. In this spatial region, the Signal-to-Noise Ratio (SNR) at the photodiode was significantly higher than the decision threshold of the LMV7219 comparator. The 4B5B encoding and the chosen On-Off Keying (OOK) modulation scheme operated well within their engineering design margins. This confirms that the discrete analog front-end is highly reliable for desktop-scale communication.

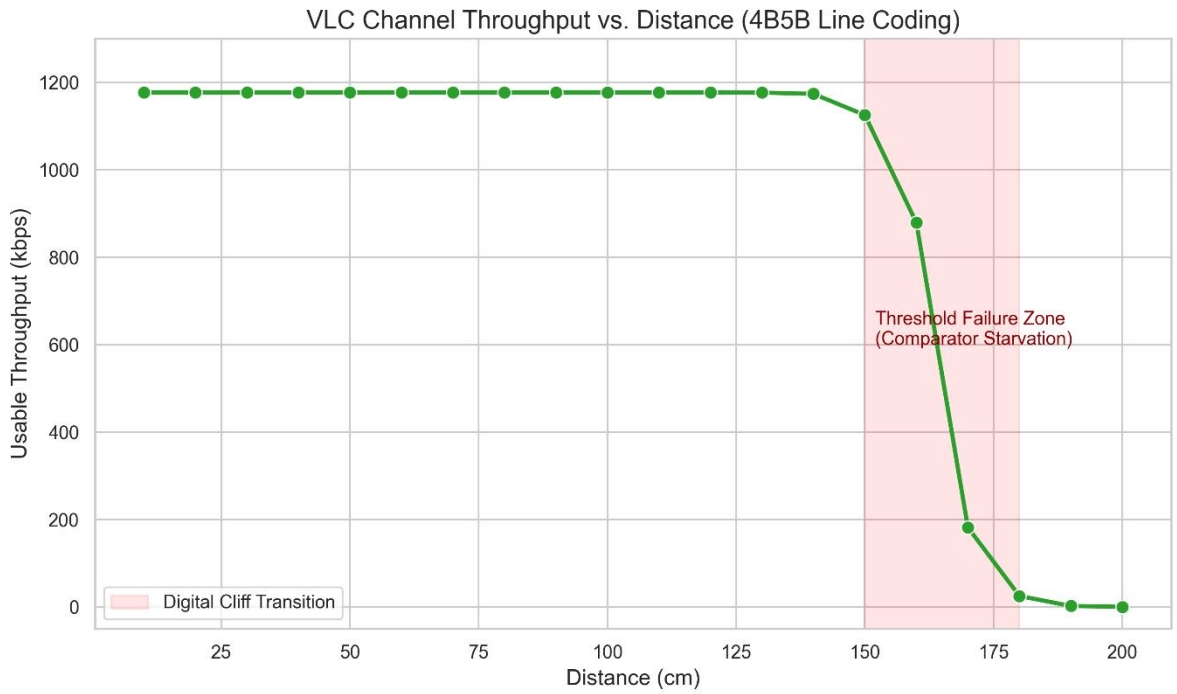

Figure 7: VLC Channel Throughput vs. Distance

4.4.2 The Transition and Degradation Zone (120 cm to 150 cm)

Starting at 120 cm, the system began to register a quantifiable emergence of bit errors. As the receiver moved further from the LED, the optical power reaching the photodiode decreased exponentially. This region represents the "knee" of the performance curve. The BER jumped significantly from the absolute noise floor up to 9.15×10^{-5} at 150 cm.

This specific boundary marks the distance where the optical channel transitioned from a deterministic, noise-free link into a stochastic, probabilistic channel. The analog voltage peaks generated by the OPA380 transimpedance amplifier began to graze the minimum detection threshold of the comparator. At this threshold, ambient room lighting and thermal noise within the circuit occasionally caused the comparator to trigger prematurely or miss a pulse entirely, resulting in the observed 4.35 percent Packet Error Rate.

4.4.3 The Collapse Zone (160 cm to 200 cm)

At 160 cm and beyond, the system reached its maximum functional link range. The PER climbed violently from 25.37 percent at 160 cm to 84.55 percent at 170 cm. In this zone, the bit errors caused by channel noise became so frequent that the software-level Cyclic Redundancy Check (CRC-16) discarded nearly all incoming packets to protect the integrity of the data stream. Consequently, the usable throughput collapsed to near zero because the receiver spent all its processing cycles discarding malformed frames rather than rendering valid data.

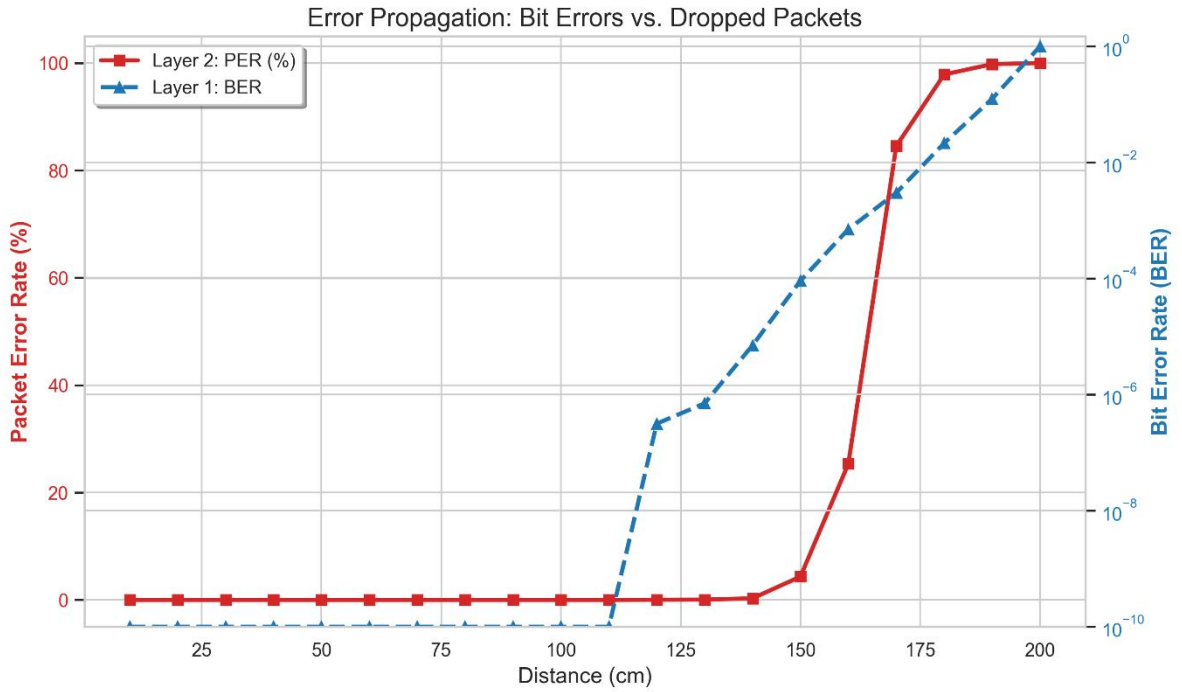

Figure 8: Bit Errors vs. Dropped Packets

4.5 Application Layer Video Performance

The empirical data directly highlights why standard video compression algorithms fail over optical links, and validates the architectural decision to utilize Motion-JPEG (M-JPEG).

At a distance of 150 cm, the system experienced a Bit Error Rate of approximately 9.15×10^{-5}. For a video stream operating at 1.17 Mbps, this error rate mathematically equates to over 100 bit errors per second. If the system had utilized a traditional inter-frame codec such as H.264, these bit errors would have corrupted the temporal reference frames (I-frames and P-frames). This would result in severe visual distortion and macro-blocking propagating across the screen for several seconds until a new reference frame arrived.

However, because M-JPEG compresses every frame as an independent image, the system achieved "Zero Temporal Propagation." When the CRC-16 algorithm dropped the corrupted packets at 150 cm, the video player simply discarded the broken frames. The video experienced minor, transient stuttering, but the visual fidelity of the successfully received frames remained perfectly intact. This proves that application-layer selection is just as critical as physical-layer design in optical wireless communications.

4.6 Discussion of Findings

The empirical data strongly validates the theoretical physics of optical wireless networks and confirms the effectiveness of the implemented software architecture.

4.6.1 Inverse-Square Law Validation

The data clearly reflects the expected physical attenuation of optical intensity over distance. According to optical physics, luminous irradiance drops off proportionally to the inverse square of the distance ($I \propto {}^{1}/_{d^2}$). The hardware performed exactly as mathematically predicted. The link maintained stability up to 110 cm while the received voltage was high. This was followed by a rapid, exponential degradation phase once the attenuated voltage crossed the critical threshold of the digital comparator, demonstrating the "Digital Cliff" phenomenon.

4.6.2 Robustness of Line Coding

The implementation of the 4B5B encoding scheme provided an exceptionally stable synchronization baseline. By ensuring that no more than three consecutive zeros were ever transmitted, the encoding prevented the receiver's baseline voltage from drifting. The error rates remained at the absolute noise floor until the physical signal power dropped below the receiver's hardware sensitivity limit at 120 cm. This proves that the observed data loss was entirely due to spatial path loss rather than a failure in baseband clock synchronization.

4.6.3 Future Improvements and Scalability

The sharp drop-off after 150 cm highlights the primary limitation of utilizing a simple voltage comparator instead of a continuous Analog-to-Digital Converter. To extend the communication range and smooth out the Digital Cliff, two logical improvements are necessary. First, implementing an adaptive Automatic Gain Control (AGC) loop would prevent the signal from falling below the comparator threshold so rapidly as the distance increases. Second, replacing the simple CRC drop mechanism with a Forward Error Correction (FEC) layer, such as a Reed-Solomon or Hamming code, would allow the receiver to mathematically reconstruct corrupted packets. This would significantly stabilize the throughput in the transition zone between 120 cm and 160 cm.

4.7 Summary

The testing validated that the discrete analog front-end and the accompanying embedded software are highly capable of sustaining real-time multimedia bandwidth. By analyzing the system from the physical voltage threshold up to the application video layer, the results confirm that the architecture behaves predictably according to the laws of optical physics. It successfully provides a reliable 1.17 Mbps video channel up to 1.1 meters before succumbing to expected path loss attenuation.

CHAPTER 5: CONCLUSION AND RECOMMENDATIONS

5.1 Introduction

This chapter concludes the research on the design and implementation of a low-cost, high-speed Visible Light Communication (VLC) system. It summarizes the primary engineering achievements relative to the initial project objectives and evaluates the viability of the discrete hardware architecture. Furthermore, it outlines specific, actionable recommendations for future hardware and software iterations to expand the capabilities of this prototype.

5.2 Conclusion

The primary objective of this project was to design, implement, and empirically evaluate a practical VLC transceiver capable of streaming High-Definition multimedia within a strict budget constraint. Based on the rigorous testing and zonal performance analysis conducted in Chapter 4, the project successfully met its core objectives.

5.2.1 Hardware Democratization and AFE Performance

The project successfully demonstrated that complex, expensive Analog-to-Digital Converters (ADCs) and Field Programmable Gate Arrays (FPGAs) are not strictly necessary for high-speed optical data transmission. By designing a highly optimized discrete Analog Front-End (AFE), the system achieved a hardware baud rate of 2 Mbps.

On the transmitter side, the integration of the TC4420 gate driver was critical in violently overcoming the parasitic capacitance of the IRLZ44N MOSFET, allowing for crisp switching of the 10W LED at nanosecond speeds. On the receiver side, the combination of the BPW34 photodiode, the OPA380 Transimpedance Amplifier, and the LMV7219 high-speed comparator successfully quantized the attenuated optical signals back into 3.3V digital logic. This proves that a sub-KES 25,000 budget is highly capable of facilitating advanced telecommunications research.

5.2.2 Protocol Efficiency and Baseband Processing

The implementation of the 4B5B line coding algorithm on the Raspberry Pi 4B baseband was highly effective. The coding scheme preserved the necessary DC-balance required for continuous, flicker-free room illumination while maintaining an 80 percent spectral efficiency. This software optimization translated the 2 Mbps physical hardware limit into a highly stable 1.17 Mbps usable payload throughput, which was perfectly sufficient for compressed multimedia.

5.2.3 Error Resilience and Multimedia Streaming

The most significant architectural success of this project was the Application-layer optimization used to combat Physical-layer noise. By discarding standard inter-frame video codecs (such as H.264) in favour of the intra-frame Motion-JPEG (M-JPEG) standard, the system achieved "Zero Temporal Propagation." When the optical channel encountered severe attenuation at the 150 cm threshold, the software successfully dropped the corrupted packets

via a CRC-16 algorithm without compromising the temporal stability of the video stream. The video player simply discarded the broken frames and instantly recovered, providing a resilient multimedia experience.

5.2.4 The Physical Limitations

While the system proved highly effective for short-range communication, the empirical data confirmed the strict limitations of the chosen architecture. The lack of an adaptive thresholding mechanism in the LMV7219 comparator resulted in a sharp "Digital Cliff." The system maintained near-perfect integrity up to 110 cm but experienced catastrophic packet loss beyond 160 cm due to standard Inverse-Square Law optical attenuation.

5.3 Recommendations for Future Work

While the current architecture successfully validates the principles of Visible Light Communication, several critical engineering enhancements are recommended for future iterations to transition the prototype into a commercially viable networking standard.

5.3.1 Hardware Recommendations: Adaptive Gain Control

The primary bottleneck limiting the transmission distance is the fixed reference voltage of the LMV7219 digital comparator. As the distance increases, the peak voltage generated by the OPA380 transimpedance amplifier falls below this fixed threshold. Future research should implement an Automatic Gain Control (AGC) loop or a Variable Gain Amplifier (VGA) immediately following the OPA380. An AGC loop would dynamically scale the amplification based on the ambient noise floor and the distance of the transmitter, effectively flattening the "Digital Cliff" and drastically extending the usable range of the receiver.

5.3.2 Software Recommendations: Forward Error Correction (FEC)

The current Data Link layer utilizes a Cyclic Redundancy Check (CRC-16) strictly for error detection. When a bit flips due to optical noise, the entire 68-byte packet is discarded. To significantly improve the effective throughput in the degradation zone (120 cm to 160 cm), future software implementations should incorporate Forward Error Correction algorithms. Implementing a Reed-Solomon or Hamming code would require slightly more processing overhead but would allow the Raspberry Pi to mathematically correct minor bit errors on the fly, salvaging packets that are currently being dropped.

5.3.3 Architectural Recommendations: Full Duplex Operation

The current prototype operates as a simplex (one-way) communication link. Modern networking requires bidirectional data transfer. Future designs should expand this architecture into a full-duplex system using Wavelength Division Multiplexing (WDM). By utilizing a cool-white LED for the downlink and an infrared (IR) or specific narrowband colour LED for the uplink, a future prototype could achieve simultaneous two-way communication without optical interference, paving the way for standard TCP/IP network integration.

5.4 Final Summary

This research successfully bridges the gap between low-speed microcontroller projects and cost-prohibitive academic VLC prototypes. By combining custom discrete analog circuitry with intelligent, error-resilient software processing, the project provides a comprehensive, tangible blueprint for democratizing high-speed optical wireless network research in resource-constrained educational environments.

References

[1] W. Wang, Z. Zeng, C. Chen, D. Wang, M. Liu and H. Haas, "UAV Array-Aided Visible Light Communication with Enhanced Angle Diversity Transmitter," *Sensors,* vol. 25, no. 18, p. 5754, 2025.

[2] V. L. Jayaweera, C. Peiris, D. Darshani, S. Edirisinghe, N. Dharmaweera and U. Wijewardhana, "Hybrid NFC-VLC Systems: Integration Strategies, Applications, and Future Directions," *Network,* vol. 5, no. 3, p. 37, 2025.

[3] F. M. Elamrawy, A. A. El Aziz, S. Khamis and H. Kasem, "Performance analysis of an indoor visible light communication system using LED configurations and diverse photodetectors," *Scientific Reports,* vol. 15, no. 1, p. 99643, 2025.

[4] R. Al Hasnawi and I. Marghescu, "A Survey of Vehicular VLC Methodologies," *Sensors,* vol. 24, no. 2, p. 598, 2024.

[5] L. Bravo-Alvarez, S. Montejo-Sánchez, L. Rodríguez-López, C. Azurdia-Meza and G. Saavedra, "A Review of Hybrid VLC/RF Networks: Features, Applications, and Future Directions," *Sensors,* vol. 23, no. 14, p. 7545, 2023.

[6] A.-M. Căilean and M. Dimian, "Increasing Vehicular Visible Light Communications Range Based on LED Current Overdriving and Variable Pulse Position Modulation: Concept and Experimental Validation," *Sensors,* vol. 23, no. 7, p. 3656, 2023.

[7] M. T. Kavlak and U. E. Ayten, "Transimpedance Amplifier Design and Performance Comparison for Photoelectric Cells," in *2023 14th International Conference on Electrical and Electronics Engineering (ELECO)*, Bursa, 2023.

[8] M. B. Othman, M. M. Ab-Rahman and M. R. Hassan, "Performance Analysis of Hybrid Pulse Amplitude Modulation and Pulse Width Modulation in VLC System," in *2021 IEEE International Conference on Computing (ICOCO)*, Kuala Lumpur, 2021.

[9] M. Z. Chowdhury, M. S. Hossan, A. Islam and Y. M. Jang, "A Comparative Study of Wireless Communication Technologies for 5G and Beyond," *IEEE Communications Surveys & Tutorials,* vol. 22, no. 3, pp. 1314-1359, 2020.

[10] H. Haas, L. Yin, C. Chen, S. Videv, D. Parol, E. Poves, H. Alshaer and M. S. Islim, "Introduction to indoor networking concepts and challenges in LiFi," *Journal of Optical Communications and Networking,* vol. 12, no. 2, pp. A190-A203, 202.

[11] S. Fuada, T. Adiono and U. Syafiqoh, "Performance Analysis of Analog Front-End Transceiver for Li-Fi in a Real-Time Audio Transmission," *Universal Journal of Electrical and Electronic Engineering,* vol. 7, no. 4, pp. 221-233, 2020.

[12] C. E. Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal,* vol. 27, no. 3, pp. 379-423, 1948.

[13] T. Komine and M. Nakagawa, “Fundamental analysis for visible-light communication system using LED lights,” *IEEE Transactions on Consumer Electronics,* vol. 50, no. 1, pp. 100-107, 2004.

[14] IEEE Computer Society. LAN/MAN Standards Committee, “IEEE Standard for Local and Metropolitan Area Networks--Part 15.7: Short-Range Wireless Optical Communication Using Visible Light,” Institute of Electrical and Electronics Engineers, 2011.

[15] G. K. Wallace, “The JPEG still picture compression standard,” *IEEE Transactions on Consumer Electronics,* vol. 38, no. 1, pp. xviii-xxxiv, 1992.

Appendix A: System Source Code

A.1 Real-Time Video Transceiver Implementation

A.1.1 Transmitter Node (TX): M-JPEG and 4B5B Encoding Algorithm

```
#include <array>
#include <cerrno>
#include <csignal>
#include <cstdint>
#include <fcntl.h>
#include <iostream>
#include <termios.h>
#include <unistd.h>
#include <vector>

constexpr const char* UART_PORT = "/dev/ttyAMA0";
constexpr size_t PAYLOAD_SIZE = 64;
constexpr int SERIAL_BAUD = B2000000;
constexpr uint8_t SYNC_WORD_1 = 0xAA;
constexpr uint8_t SYNC_WORD_2 = 0xD4;

/* ANSI X3T9.5 4B5B Encoding Table (Prioritizes DC Balance and Clock Transitions) */
const uint8_t MAP_4B5B[16] = {
    0x1E, 0x09, 0x14, 0x15, 0x0A, 0x0B, 0x12, 0x13,
    0x0E, 0x0F, 0x16, 0x17, 0x1C, 0x1D, 0x1A, 0x1B
};

/* * Encodes exactly 4 raw bytes (32 bits) into 5 transmitted bytes (40 bits).
 * Time Complexity: O(1) per block. Space: O(1).
 */
void encode_4b5b_block(const uint8_t* in4, uint8_t* out5) {
    uint64_t accum = 0;
    for (int i = 0; i < 4; ++i) {
        uint8_t hi = (in4[i] >> 4) & 0x0F;
        uint8_t lo = in4[i] & 0x0F;
        accum = (accum << 5) | MAP_4B5B[hi];
        accum = (accum << 5) | MAP_4B5B[lo];
    }

    // Unpack 40-bit accumulator into 5 distinct 8-bit UART bytes
    out5[0] = static_cast<uint8_t>((accum >> 32) & 0xFF);
    out5[1] = static_cast<uint8_t>((accum >> 24) & 0xFF);
    out5[2] = static_cast<uint8_t>((accum >> 16) & 0xFF);
    out5[3] = static_cast<uint8_t>((accum >> 8) & 0xFF);
    out5[4] = static_cast<uint8_t>(accum & 0xFF);
}

uint16_t calculate_crc16(const std::vector<uint8_t>& data) {
    uint16_t crc = 0xFFFF;
    for (uint8_t byte : data) {
        crc ^= static_cast<uint16_t>(byte) << 8;
        for (int i = 0; i < 8; ++i) {
            crc = (crc & 0x8000) ? (crc << 1) ^ 0x1021 : (crc << 1);
        }
    }
    return crc;
}

ssize_t read_all(int fd, uint8_t* buffer, size_t count) {
    size_t total = 0;
    while (total < count) {
```

```
57.         ssize_t result = read(fd, buffer + total, count - total);
58.         if (result < 0) {
59.             if (errno == EINTR) continue;
60.             return -1;
61.         }
62.         if (result == 0) break;
63.         total += static_cast<size_t>(result);
64.     }
65.     return static_cast<ssize_t>(total);
66. }
67.
68. bool write_all(int fd, const uint8_t* buffer, size_t count) {
69.     size_t total = 0;
70.     while (total < count) {
71.         ssize_t result = write(fd, buffer + total, count - total);
72.         if (result < 0) {
73.             if (errno == EINTR) continue;
74.             return false;
75.         }
76.         total += static_cast<size_t>(result);
77.     }
78.     return true;
79. }
80.
81. int setup_uart() {
82.     int fd = open(UART_PORT, O_RDWR | O_NOCTTY);
83.     if (fd == -1) return -1;
84.
85.     struct termios options;
86.     if (tcgetattr(fd, &options) != 0) return -1;
87.
88.     cfsetispeed(&options, SERIAL_BAUD);
89.     cfsetospeed(&options, SERIAL_BAUD);
90.
91.     options.c_cflag &= ~PARENB;
92.     options.c_cflag &= ~CSTOPB;
93.     options.c_cflag &= ~CSIZE;
94.     options.c_cflag |= CS8 | CREAD | CLOCAL;
95.
96.     options.c_iflag &= ~(IXON | IXOFF | IXANY | BRKINT | INPCK | ISTRIP | IGNCR | ICRNL);
97.     options.c_oflag &= ~OPOST;
98.     options.c_lflag &= ~(ICANON | ECHO | ECHOE | ISIG | IEXTEN);
99.     options.c_cc[VMIN] = 0;
100.    options.c_cc[VTIME] = 0;
101.
102.    tcflush(fd, TCIOFLUSH);
103.    if (tcsetattr(fd, TCSANOW, &options) != 0) return -1;
104.
105.    return fd;
106. }
107.
108. int main() {
109.     std::signal(SIGPIPE, SIG_IGN);
110.     int serial_fd = setup_uart();
111.     if (serial_fd == -1) {
112.         std::cerr << "[TX] Failed to open UART port\n";
113.         return 1;
114.     }
115.
116.     std::cerr << "[TX] 2Mbps 4B5B Transmitter Ready. (Theoretical Throughput: 1.6 Mbps)\n";
117.
118.     uint8_t stdin_buf[PAYLOAD_SIZE] = {};
119.     uint8_t sequence = 0;
120.
```

```
121.     while (true) {
122.         ssize_t bytes_read = read_all(STDIN_FILENO, stdin_buf, PAYLOAD_SIZE);
123.         if (bytes_read <= 0) break;
124.
125.         std::vector<uint8_t> raw_packet;
126.         raw_packet.reserve(2 + PAYLOAD_SIZE + 2); // Len(1) + Seq(1) + Payload(64) + CRC(2)
= 68 bytes
127.
128.         raw_packet.push_back(static_cast<uint8_t>(bytes_read));
129.         raw_packet.push_back(sequence++);
130.         raw_packet.insert(raw_packet.end(), stdin_buf, stdin_buf + bytes_read);
131.
132.         if (bytes_read < static_cast<ssize_t>(PAYLOAD_SIZE)) {
133.             raw_packet.insert(raw_packet.end(), PAYLOAD_SIZE - bytes_read, 0x00);
134.         }
135.
136.         uint16_t crc = calculate_crc16(raw_packet);
137.         raw_packet.push_back(static_cast<uint8_t>((crc >> 8) & 0xFF));
138.         raw_packet.push_back(static_cast<uint8_t>(crc & 0xFF));
139.
140.         // 68 bytes raw / 4 = 17 blocks. 17 * 5 = 85 bytes encoded.
141.         std::vector<uint8_t> tx_buffer;
142.         tx_buffer.reserve(2 + 85);
143.
144.         tx_buffer.push_back(SYNC_WORD_1);
145.         tx_buffer.push_back(SYNC_WORD_2);
146.
147.         // Apply 4B5B compression algorithm block-by-block
148.         for (size_t i = 0; i < raw_packet.size(); i += 4) {
149.             uint8_t out5[5];
150.             encode_4b5b_block(&raw_packet[i], out5);
151.             tx_buffer.insert(tx_buffer.end(), out5, out5 + 5);
152.         }
153.
154.         if (!write_all(serial_fd, tx_buffer.data(), tx_buffer.size())) break;
155.     }
156.
157.     close(serial_fd);
158.     return 0;
159. }
160.
```

A.1.2 Receiver Node (RX): 4B5B Decoding and Video Rendering Algorithm

```
 1. #include <iostream>
 2. #include <vector>
 3. #include <cstring>
 4. #include <fcntl.h>
 5. #include <unistd.h>
 6. #include <termios.h>
 7. #include <cstdint>
 8. #include <array>
 9.
10. constexpr const char* DEFAULT_UART_PORT = "/dev/ttyAMA0";
11. constexpr size_t PAYLOAD_SIZE = 64;
12. constexpr int SERIAL_BAUD = B2000000;
13. constexpr uint8_t SYNC_WORD_1 = 0xAA;
14. constexpr uint8_t SYNC_WORD_2 = 0xD4;
15. constexpr size_t ENCODED_SIZE = 85; // 17 blocks * 5 bytes
16.
17. enum RX_State { WAIT_SYNC1, WAIT_SYNC2, GET_ENCODED };
18.
19. std::array<uint8_t, 32> generate_reverse_4b5b() {
```

```
20.     std::array<uint8_t, 32> rev;
21.     rev.fill(0xFF); // 0xFF marks an illegal channel-noise symbol
22.     const uint8_t map[16] = {
23.         0x1E, 0x09, 0x14, 0x15, 0x0A, 0x0B, 0x12, 0x13,
24.         0x0E, 0x0F, 0x16, 0x17, 0x1C, 0x1D, 0x1A, 0x1B
25.     };
26.     for (uint8_t i = 0; i < 16; ++i) rev[map[i]] = i;
27.     return rev;
28. }
29.
30. /* * Decodes exactly 5 transmitted bytes (40 bits) back into 4 raw bytes (32 bits).
31.  * Returns false if an illegal 5-bit optical noise symbol is detected.
32.  */
33. bool decode_4b5b_block(const uint8_t* in5, uint8_t* out4, const std::array<uint8_t, 32>&
rev_map) {
34.     uint64_t accum = 0;
35.     accum |= (static_cast<uint64_t>(in5[0]) << 32);
36.     accum |= (static_cast<uint64_t>(in5[1]) << 24);
37.     accum |= (static_cast<uint64_t>(in5[2]) << 16);
38.     accum |= (static_cast<uint64_t>(in5[3]) << 8);
39.     accum |= (static_cast<uint64_t>(in5[4]));
40.
41.     for (int i = 0; i < 4; ++i) {
42.         uint8_t hi_sym = (accum >> (35 - i * 10)) & 0x1F;
43.         uint8_t lo_sym = (accum >> (30 - i * 10)) & 0x1F;
44.
45.         uint8_t hi_nibble = rev_map[hi_sym];
46.         uint8_t lo_nibble = rev_map[lo_sym];
47.
48.         if (hi_nibble == 0xFF || lo_nibble == 0xFF) return false; // Channel Noise Detected!
49.         out4[i] = (hi_nibble << 4) | lo_nibble;
50.     }
51.     return true;
52. }
53.
54. uint16_t calculate_crc16(const std::vector<uint8_t>& data) {
55.     uint16_t crc = 0xFFFF;
56.     for (uint8_t byte : data) {
57.         crc ^= static_cast<uint16_t>(byte) << 8;
58.         for (int i = 0; i < 8; i++) {
59.             crc = (crc & 0x8000) ? (crc << 1) ^ 0x1021 : (crc << 1);
60.         }
61.     }
62.     return crc;
63. }
64.
65. int setup_uart(const char* port) {
66.     int fd = open(port, O_RDONLY | O_NOCTTY);
67.     if (fd == -1) return -1;
68.
69.     struct termios options;
70.     if (tcgetattr(fd, &options) == -1) return -1;
71.     if (cfsetispeed(&options, SERIAL_BAUD) == -1 || cfsetospeed(&options, SERIAL_BAUD) == -
1) return -1;
72.
73.     options.c_cflag &= ~PARENB;
74.     options.c_cflag &= ~CSTOPB;
75.     options.c_cflag &= ~CSIZE;
76.     options.c_cflag |= CS8 | CREAD | CLOCAL;
77.     options.c_lflag &= ~(ICANON | ECHO | ECHOE | ISIG);
78.     options.c_iflag &= ~(IXON | IXOFF | IXANY);
79.     options.c_oflag &= ~OPOST;
80.
81.     if (tcsetattr(fd, TCSANOW, &options) == -1) return -1;
```

```
    return fd;
}

int main(int argc, char* argv[]) {
    const char* uart_port = (argc > 1) ? argv[1] : DEFAULT_UART_PORT;
    const auto rev_4b5b_map = generate_reverse_4b5b();
    int serial_fd = setup_uart(uart_port);
    if (serial_fd == -1) return 1;

    std::cerr << "[RX] 2Mbps 4B5B Receiver Ready.\n";

    RX_State state = WAIT_SYNC1;
    std::vector<uint8_t> encoded_buffer;
    encoded_buffer.reserve(ENCODED_SIZE);

    uint8_t read_buf[4096];

    while (true) {
        ssize_t bytes_read = read(serial_fd, read_buf, sizeof(read_buf));
        if (bytes_read <= 0) continue;

        for (ssize_t i = 0; i < bytes_read; ++i) {
            uint8_t byte = read_buf[i];

            switch (state) {
                case WAIT_SYNC1:
                    if (byte == SYNC_WORD_1) state = WAIT_SYNC2;
                    break;
                case WAIT_SYNC2:
                    if (byte == SYNC_WORD_2) {
                        encoded_buffer.clear();
                        state = GET_ENCODED;
                    } else {
                        state = (byte == SYNC_WORD_1) ? WAIT_SYNC2 : WAIT_SYNC1;
                    }
                    break;
                case GET_ENCODED:
                    encoded_buffer.push_back(byte);

                    if (encoded_buffer.size() == ENCODED_SIZE) {
                        std::vector<uint8_t> raw_packet;
                        raw_packet.reserve(68);
                        bool decode_success = true;

                        // Decode blocks of 5 bytes back to 4 bytes
                        for (size_t j = 0; j < ENCODED_SIZE; j += 5) {
                            uint8_t out4[4];
                            if (!decode_4b5b_block(&encoded_buffer[j], out4, rev_4b5b_map))
{
                                decode_success = false;
                                break;
                            }
                            raw_packet.insert(raw_packet.end(), out4, out4 + 4);
                        }

                        if (decode_success) {
                            // Extract headers and CRC
                            uint8_t payload_len = raw_packet[0];
                            uint16_t received_crc = (static_cast<uint16_t>(raw_packet[66])
<< 8) | raw_packet[67];

                            // Strip CRC for verification
                            raw_packet.pop_back();
                            raw_packet.pop_back();
```

```
144.
145.                             if (calculate_crc16(raw_packet) == received_crc) {
146.                                 // Robust output for M-JPEG pipeline
147.                                 write(STDOUT_FILENO, raw_packet.data() + 2, payload_len);
148.                             } else {
149.                                 decode_success = false;
150.                             }
151.                         }
152.
153.                         // ERROR HANDLING: Zero Padding for M-JPEG Continuity
154.                         if (!decode_success) {
155.                             std::cerr << "[RX] 4B5B/CRC Error. Zero-padding to maintain
stream continuity.\n";
156.                             std::vector<uint8_t> dummy_pad(PAYLOAD_SIZE, 0x00);
157.                             write(STDOUT_FILENO, dummy_pad.data(), dummy_pad.size());
158.                         }
159.
160.                         state = WAIT_SYNC1;
161.                     }
162.                     break;
163.                 }
164.             }
165.         }
166.         return 0;
167. }
168.
```

A.2 Physical Link Characterization and Benchmarking

A.2.1 Transmitter Node (TX): Golden Payload Generator

```
 1. #include <array>
 2. #include <cerrno>
 3. #include <csignal>
 4. #include <cstdint>
 5. #include <fcntl.h>
 6. #include <iostream>
 7. #include <termios.h>
 8. #include <unistd.h>
 9. #include <vector>
10.
11. constexpr const char* UART_PORT = "/dev/ttyAMA0";
12. constexpr size_t PAYLOAD_SIZE = 64;
13. constexpr int SERIAL_BAUD = B2000000; // 2 Mbps
14. constexpr uint8_t SYNC_WORD_1 = 0xAA;
15. constexpr uint8_t SYNC_WORD_2 = 0xD4;
16.
17. /* ANSI X3T9.5 4B5B Encoding Table */
18. const uint8_t MAP_4B5B[16] = {
19.     0x1E, 0x09, 0x14, 0x15, 0x0A, 0x0B, 0x12, 0x13,
20.     0x0E, 0x0F, 0x16, 0x17, 0x1C, 0x1D, 0x1A, 0x1B
21. };
22.
23. void encode_4b5b_block(const uint8_t* in4, uint8_t* out5) {
24.     uint64_t accum = 0;
25.     for (int i = 0; i < 4; ++i) {
26.         uint8_t hi = (in4[i] >> 4) & 0x0F;
27.         uint8_t lo = in4[i] & 0x0F;
28.         accum = (accum << 5) | MAP_4B5B[hi];
29.         accum = (accum << 5) | MAP_4B5B[lo];
30.     }
31.     out5[0] = static_cast<uint8_t>((accum >> 32) & 0xFF);
32.     out5[1] = static_cast<uint8_t>((accum >> 24) & 0xFF);
```

```
    out5[2] = static_cast<uint8_t>((accum >> 16) & 0xFF);
    out5[3] = static_cast<uint8_t>((accum >> 8) & 0xFF);
    out5[4] = static_cast<uint8_t>(accum & 0xFF);
}

uint16_t calculate_crc16(const std::vector<uint8_t>& data) {
    uint16_t crc = 0xFFFF;
    for (uint8_t byte : data) {
        crc ^= static_cast<uint16_t>(byte) << 8;
        for (int i = 0; i < 8; ++i) {
            crc = (crc & 0x8000) ? (crc << 1) ^ 0x1021 : (crc << 1);
        }
    }
    return crc;
}

bool write_all(int fd, const uint8_t* buffer, size_t count) {
    size_t total = 0;
    while (total < count) {
        ssize_t result = write(fd, buffer + total, count - total);
        if (result < 0) {
            if (errno == EINTR) continue;
            return false;
        }
        total += static_cast<size_t>(result);
    }
    return true;
}

int setup_uart() {
    int fd = open(UART_PORT, O_RDWR | O_NOCTTY);
    if (fd == -1) return -1;
    struct termios options;
    if (tcgetattr(fd, &options) != 0) return -1;
    cfsetispeed(&options, SERIAL_BAUD);
    cfsetospeed(&options, SERIAL_BAUD);
    options.c_cflag &= ~PARENB;
    options.c_cflag &= ~CSTOPB;
    options.c_cflag &= ~CSIZE;
    options.c_cflag |= CS8 | CREAD | CLOCAL;
    options.c_iflag &= ~(IXON | IXOFF | IXANY | BRKINT | INPCK | ISTRIP | IGNCR | ICRNL);
    options.c_oflag &= ~OPOST;
    options.c_lflag &= ~(ICANON | ECHO | ECHOE | ISIG | IEXTEN);
    options.c_cc[VMIN] = 0;
    options.c_cc[VTIME] = 0;
    tcflush(fd, TCIOFLUSH);
    if (tcsetattr(fd, TCSANOW, &options) != 0) return -1;
    return fd;
}

int main() {
    std::signal(SIGPIPE, SIG_IGN);
    int serial_fd = setup_uart();
    if (serial_fd == -1) {
        std::cerr << "[TX] Failed to open UART port\n";
        return 1;
    }

    std::cerr << "[TX] 3-Minute Benchmark Transmitter Ready. Blasting data...\n";

    // Create the "Golden Payload" - e.g., an alternating bit pattern 0x55 (01010101)
    uint8_t golden_payload[PAYLOAD_SIZE];
    for(size_t i = 0; i < PAYLOAD_SIZE; ++i) {
        golden_payload[i] = 0x55;
```

```
    }

    uint8_t sequence = 0;

    while (true) {
        std::vector<uint8_t> raw_packet;
        raw_packet.reserve(2 + PAYLOAD_SIZE + 2);

        raw_packet.push_back(static_cast<uint8_t>(PAYLOAD_SIZE));
        raw_packet.push_back(sequence++);
        raw_packet.insert(raw_packet.end(), golden_payload, golden_payload + PAYLOAD_SIZE);

        uint16_t crc = calculate_crc16(raw_packet);
        raw_packet.push_back(static_cast<uint8_t>((crc >> 8) & 0xFF));
        raw_packet.push_back(static_cast<uint8_t>(crc & 0xFF));

        std::vector<uint8_t> tx_buffer;
        tx_buffer.reserve(2 + 85);
        tx_buffer.push_back(SYNC_WORD_1);
        tx_buffer.push_back(SYNC_WORD_2);

        for (size_t i = 0; i < raw_packet.size(); i += 4) {
            uint8_t out5[5];
            encode_4b5b_block(&raw_packet[i], out5);
            tx_buffer.insert(tx_buffer.end(), out5, out5 + 5);
        }

        if (!write_all(serial_fd, tx_buffer.data(), tx_buffer.size())) break;
    }

    close(serial_fd);
    return 0;
}

```

A.2.2 Receiver Node (RX): BER, PER, and Throughput Calculation Script

```
#include <iostream>
#include <vector>
#include <cstring>
#include <fcntl.h>
#include <unistd.h>
#include <termios.h>
#include <cstdint>
#include <array>
#include <chrono>
#include <iomanip>

constexpr const char* DEFAULT_UART_PORT = "/dev/ttyAMA0";
constexpr size_t PAYLOAD_SIZE = 64;
constexpr int SERIAL_BAUD = B2000000;
constexpr uint8_t SYNC_WORD_1 = 0xAA;
constexpr uint8_t SYNC_WORD_2 = 0xD4;
constexpr size_t ENCODED_SIZE = 85;

enum RX_State { WAIT_SYNC1, WAIT_SYNC2, GET_ENCODED };

std::array<uint8_t, 32> generate_reverse_4b5b() {
    std::array<uint8_t, 32> rev;
    rev.fill(0xFF);
    const uint8_t map[16] = {
        0x1E, 0x09, 0x14, 0x15, 0x0A, 0x0B, 0x12, 0x13,
        0x0E, 0x0F, 0x16, 0x17, 0x1C, 0x1D, 0x1A, 0x1B
    };
    for (uint8_t i = 0; i < 16; ++i) rev[map[i]] = i;
```

```
    return rev;
}

bool decode_4b5b_block(const uint8_t* in5, uint8_t* out4, const std::array<uint8_t, 32>&
rev_map) {
    uint64_t accum = 0;
    accum |= (static_cast<uint64_t>(in5[0]) << 32);
    accum |= (static_cast<uint64_t>(in5[1]) << 24);
    accum |= (static_cast<uint64_t>(in5[2]) << 16);
    accum |= (static_cast<uint64_t>(in5[3]) << 8);
    accum |= (static_cast<uint64_t>(in5[4]));

    for (int i = 0; i < 4; ++i) {
        uint8_t hi_sym = (accum >> (35 - i * 10)) & 0x1F;
        uint8_t lo_sym = (accum >> (30 - i * 10)) & 0x1F;
        uint8_t hi_nibble = rev_map[hi_sym];
        uint8_t lo_nibble = rev_map[lo_sym];

        if (hi_nibble == 0xFF || lo_nibble == 0xFF) return false;
        out4[i] = (hi_nibble << 4) | lo_nibble;
    }
    return true;
}

uint16_t calculate_crc16(const std::vector<uint8_t>& data) {
    uint16_t crc = 0xFFFF;
    for (uint8_t byte : data) {
        crc ^= static_cast<uint16_t>(byte) << 8;
        for (int i = 0; i < 8; i++) {
            crc = (crc & 0x8000) ? (crc << 1) ^ 0x1021 : (crc << 1);
        }
    }
    return crc;
}

/* Kernighan's Algorithm to count bit errors in O(N) time */
int count_bit_errors(const uint8_t* received, const uint8_t* expected, size_t len) {
    int errors = 0;
    for (size_t i = 0; i < len; ++i) {
        uint8_t diff = received[i] ^ expected[i];
        while (diff) {
            diff &= (diff - 1);
            errors++;
        }
    }
    return errors;
}

int setup_uart(const char* port) {
    int fd = open(port, O_RDONLY | O_NOCTTY);
    if (fd == -1) return -1;
    struct termios options;
    if (tcgetattr(fd, &options) == -1) return -1;
    if (cfsetispeed(&options, SERIAL_BAUD) == -1 || cfsetospeed(&options, SERIAL_BAUD) == -
1) return -1;
    options.c_cflag &= ~PARENB;
    options.c_cflag &= ~CSTOPB;
    options.c_cflag &= ~CSIZE;
    options.c_cflag |= CS8 | CREAD | CLOCAL;
    options.c_lflag &= ~(ICANON | ECHO | ECHOE | ISIG);
    options.c_iflag &= ~(IXON | IXOFF | IXANY);
    options.c_oflag &= ~OPOST;

```

```
    // Set a 0.1 second timeout so read() doesn't block forever if the laser is
disconnected
    options.c_cc[VMIN] = 0;
    options.c_cc[VTIME] = 1;

    if (tcsetattr(fd, TCSANOW, &options) == -1) return -1;
    return fd;
}

int main(int argc, char* argv[]) {
    const char* uart_port = (argc > 1) ? argv[1] : DEFAULT_UART_PORT;
    const auto rev_4b5b_map = generate_reverse_4b5b();
    int serial_fd = setup_uart(uart_port);
    if (serial_fd == -1) return 1;

    std::cerr << "[RX] Waiting for laser connection...\n";

    RX_State state = WAIT_SYNC1;
    std::vector<uint8_t> encoded_buffer;
    encoded_buffer.reserve(ENCODED_SIZE);
    uint8_t read_buf[4096];

    // The Golden Payload for BER comparison
    uint8_t golden_payload[PAYLOAD_SIZE];
    for(size_t i = 0; i < PAYLOAD_SIZE; ++i) golden_payload[i] = 0x55;

    // Metrics Tracking
    uint64_t total_payload_bits = 0;
    uint64_t bit_errors = 0;
    uint32_t good_packets = 0;
    uint32_t corrupted_packets = 0;
    uint32_t lost_packets = 0;
    int32_t last_seq = -1;

    bool test_started = false;
    auto start_time = std::chrono::steady_clock::now();
    const double TEST_DURATION_SECONDS = 180.0; // 3 minutes

    while (true) {
        if (test_started) {
            auto current_time = std::chrono::steady_clock::now();
            std::chrono::duration<double> elapsed = current_time - start_time;
            if (elapsed.count() >= TEST_DURATION_SECONDS) {
                break; // 3 minutes are up!
            }
        }

        ssize_t bytes_read = read(serial_fd, read_buf, sizeof(read_buf));
        if (bytes_read <= 0) continue;

        for (ssize_t i = 0; i < bytes_read; ++i) {
            uint8_t byte = read_buf[i];

            switch (state) {
                case WAIT_SYNC1:
                    if (byte == SYNC_WORD_1) state = WAIT_SYNC2;
                    break;
                case WAIT_SYNC2:
                    if (byte == SYNC_WORD_2) {
                        encoded_buffer.clear();
                        state = GET_ENCODED;
                    } else {
                        state = (byte == SYNC_WORD_1) ? WAIT_SYNC2 : WAIT_SYNC1;
                    }
```

```
153.                     break;
154.                 case GET_ENCODED:
155.                     encoded_buffer.push_back(byte);
156.
157.                     if (encoded_buffer.size() == ENCODED_SIZE) {
158.                         if (!test_started) {
159.                             std::cerr << "[RX] Connection Acquired! Starting 3-minute
test...\n";
160.                             start_time = std::chrono::steady_clock::now();
161.                             test_started = true;
162.                         }
163.
164.                         std::vector<uint8_t> raw_packet;
165.                         raw_packet.reserve(68);
166.                         bool decode_success = true;
167.
168.                         for (size_t j = 0; j < ENCODED_SIZE; j += 5) {
169.                             uint8_t out4[4];
170.                             if (!decode_4b5b_block(&encoded_buffer[j], out4, rev_4b5b_map))
{
171.                                 decode_success = false;
172.                                 break;
173.                             }
174.                             raw_packet.insert(raw_packet.end(), out4, out4 + 4);
175.                         }
176.
177.                         if (decode_success) {
178.                             uint8_t seq = raw_packet[1];
179.                             uint16_t received_crc = (static_cast<uint16_t>(raw_packet[66])
<< 8) | raw_packet[67];
180.
181.                             // Check for lost packets using sequence number jump
182.                             if (last_seq != -1) {
183.                                 int diff = (seq - last_seq) & 0xFF; // Accounts for 0-255
rollover
184.                                 if (diff > 1) {
185.                                     lost_packets += (diff - 1);
186.                                 }
187.                             }
188.                             last_seq = seq;
189.
190.                             raw_packet.pop_back();
191.                             raw_packet.pop_back();
192.
193.                             if (calculate_crc16(raw_packet) == received_crc) {
194.                                 good_packets++;
195.                             } else {
196.                                 corrupted_packets++;
197.                             }
198.
199.                             // Calculate BER on the payload (Bytes 2 through 65)
200.                             int errors = count_bit_errors(raw_packet.data() + 2,
golden_payload, PAYLOAD_SIZE);
201.                             bit_errors += errors;
202.                             total_payload_bits += (PAYLOAD_SIZE * 8);
203.
204.                         } else {
205.                             // If 4B5B fails, the packet is corrupted by physical channel
noise
206.                             corrupted_packets++;
207.                             // Sequence number is unreadable, assume it increments by 1
208.                             if (last_seq != -1) last_seq = (last_seq + 1) & 0xFF;
209.                         }
210.
```

```
                        state = WAIT_SYNC1;
                    }
                    break;
            }
        }
    }

    // --- MATHEMATICAL ANALYSIS & CSV GENERATION ---
    uint32_t total_expected_packets = good_packets + corrupted_packets + lost_packets;
    double per_percentage = (total_expected_packets == 0) ? 100.0 :
        (static_cast<double>(corrupted_packets + lost_packets) / total_expected_packets) *
100.0;

    double ber = (total_payload_bits == 0) ? 1.0 :
        static_cast<double>(bit_errors) / static_cast<double>(total_payload_bits);

    // Throughput formula: (Good Packets * Payload Bits) / Total Time
    double throughput_bps = (static_cast<double>(good_packets) * PAYLOAD_SIZE * 8.0) /
TEST_DURATION_SECONDS;
    double throughput_kbps = throughput_bps / 1000.0;

    std::cerr << "\n[TEST COMPLETE] Raw Data Collection Generated.\n";
    std::cerr << "Distance (cm),Ambient Light (Lux),Total Bits,Bit Errors,BER,Total
Packets,Lost Packets,PER (%),Throughput (kbps)\n";

    std::cout << std::fixed << std::setprecision(8);
    std::cout << "ENTER_DIST,ENTER_LUX,"
              << total_payload_bits << ","
              << bit_errors << ","
              << ber << ","
              << total_expected_packets << ","
              << (corrupted_packets + lost_packets) << ","
              << std::setprecision(4) << per_percentage << ","
              << throughput_kbps << "\n";

    return 0;
}
```

Appendix B: Bill of Materials and Budget

The financial breakdown for the prototype development is detailed in the table below. The budget is explicitly partitioned into the Transmitter (TX) and Receiver (RX) subsystems to reflect the discrete hardware architecture.

Table 2: Bill of Materials and Project Budget

Item Category	Component Specification	Qty	Unit Cost (KES)	Total (KES)
TRANSMITTER SUBSYSTEM (TX)				
Processing Unit	Raspberry Pi 4 Model B (2GB RAM)	1	9,500	9,500
Optical Source	10W High-Power LED (Cool White) + Heatsink	1	450	450
Switching Matrix	TC4420 High-Speed Driver IC	1	300	300
Power Switching	IRFZ44N Logic-Level MOSFET	2	150	300
Power Supply	12V 5A DC SMPS Adapter	1	800	800
TX Subsystem Estimate				**11,350**
RECEIVER SUBSYSTEM (RX)				
Processing / Sink	Raspberry Pi 4 Model B (2GB RAM)	1	9,500	9,500
Optical Sensor	BPW34 Silicon PIN Photodiode	2	150	300

Analog Front-End	OPA380 Precision High-Speed TIA	1	1,200	1,200
Quantization Stage	LMV7219 High-Speed Voltage Comparator	1	350	350
Passives & Prototyping	Copper-clad boards, jumper cables, and precision resistors	Lot	800	800
RX Subsystem Estimate				**12,150**
GRAND TOTAL	**Project Budget**			**23,500**

Appendix C: Project Timeline and Gantt Chart

The development of the VLC transceiver followed a strict systems-engineering methodology. The project was executed over a 20-week schedule, categorized into Research, Hardware Fabrication, Software Development, System Integration, and Empirical Benchmarking.

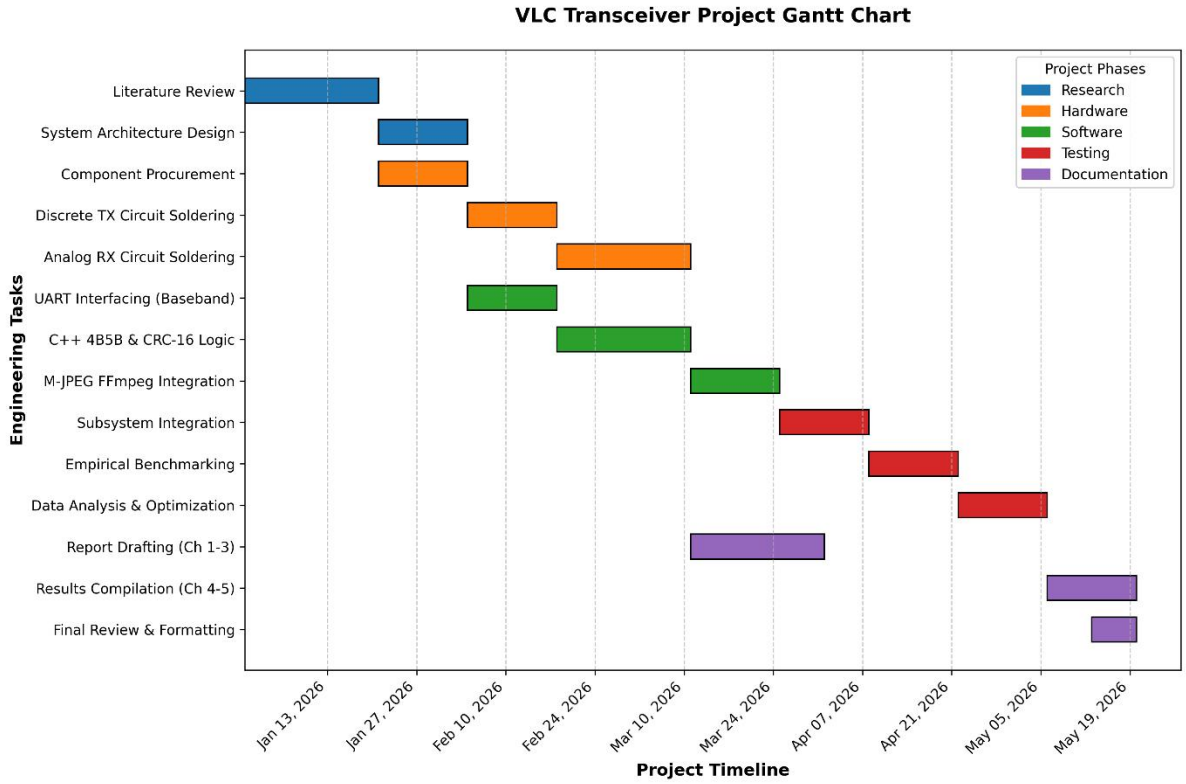

Figure 9: VLC Transceiver Project Gantt Chart